First program ChatGPT

a. Name of two GenAI programs: **ChatGPT, Claude code**

b. How you accessed them (e.g., Google query, CoPilot on my computer): **My openAI account web interface and anthropic account web interface**

c. Prompt used. (You must use the same prompt for both GenAIs.)

prompt used:

MAke a program that follows the following assignment. the file should be named Assignment4.cpp udoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-

grids, or blocks.

• A typical puzzle is shown here (image credit: Wikimedia Commons):

o

• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject

to the following constraints:

o Each digit from 1 to 9 must appear exactly once in each row.

o Each digit from 1 to 9 must appear exactly once in each column.

o Each digit from 1 to 9 must appear exactly once in each block.

• Published puzzles are usually constructed to have a unique solution. This

requires that at least 17 cells be filled in (and may require more; depending on

the puzzle and which cells are empty, as many as 78 filled cells can be

required to guarantee uniqueness). In general, the number of cells that are

already filled in is not a particularly good indicator of how difficult a Sudoku

puzzle is to solve.

• The Sudoku problem, then, is this: Given a partially completed Sudoku

puzzle, find the (or at least a) solution; if no solution exists, prove that.

• It is possible to solve a Sudoku puzzle entirely through logic. For example, in

the above puzzle, consider the bottom right block, top right cell. Because of

other numbers in the same row, column, or grid, that number cannot be 1, 2, 3,

5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these

and more advanced deduction rules, it's possible to gradually fill in the entire

grid.

• That is not the approach we're going to use. Instead, we're going to

implement a brute-force search, in which we exhaustively try possibilities

until we find a solution. This will be slower to run but is easier to solve. (And

by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)

• Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.

• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.

• Write a recursive object-oriented program to find the solution to the five puzzle files.

• The output for each solution, should include:

o The puzzle file name (e.g., puzzle1.txt)

o The puzzle from the file printed out.

o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.

• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.

• To help you debug your program, solution1.txt is a solution to puzzle1.txt.

Chat GPT

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iomanip>

using namespace std;

class SudokuSolver {
private:
    int grid[9][9];
    int original[9][9];
    int solutionCount;

    bool isValid(int row, int col, int value) const {
        // check the row
        for (int c = 0; c < 9; c++) {
            if (grid[row][c] == value) {
                return false;
            }
        }

        // check the column
        for (int r = 0; r < 9; r++) {
            if (grid[r][col] == value) {
                return false;
            }
        }

        // check the 3x3 block
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                if (grid[r][c] == value) {
                    return false;
                }
```

```
        }
    }

    return true;
}

bool initialPuzzleIsValid() const {
    // Check every filled cell against the other filled cells.
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (grid[row][col] != 0) {
                int value = grid[row][col];

                // check the row
                for (int c = 0; c < 9; c++) {
                    if (c != col && grid[row][c] == value) {
                        return false;
                    }
                }

                // check the column
                for (int r = 0; r < 9; r++) {
                    if (r != row && grid[r][col] == value) {
                        return false;
                    }
                }

                // check the block
                int startRow = (row / 3) * 3;
                int startCol = (col / 3) * 3;

                for (int r = startRow; r < startRow + 3; r++) {
                    for (int c = startCol; c < startCol + 3; c++) {
                        if ((r != row || c != col) &&
                            grid[r][c] == value) {
                            return false;
                        }
                    }
```

```
            }
        }
    }
}

    return true;
}

// Recursive depth-first search with backtracking.
void solveRecursive() {
    // Find the next empty cell.
    int emptyRow = -1;
    int emptyCol = -1;

    for (int row = 0; row < 9 && emptyRow == -1; row++) {
        for (int col = 0; col < 9; col++) {
            if (grid[row][col] == 0) {
                emptyRow = row;
                emptyCol = col;
                break;
            }
        }
    }

    // No empty cells means the current grid is a complete solution.
    if (emptyRow == -1) {
        solutionCount++;
        printGrid();
        return;
    }

    // Try every possible value in the empty cell.
    for (int value = 1; value <= 9; value++) {
        if (isValid(emptyRow, emptyCol, value)) {
            grid[emptyRow][emptyCol] = value;

            // Recursively try to solve the rest of the puzzle.
            solveRecursive();
```

```
                // Undo the choice so another possibility can be tried.
                grid[emptyRow][emptyCol] = 0;
            }
        }
    }

    void printGrid() const {
        cout << "+-------+-------+-------+" << endl;

        for (int row = 0; row < 9; row++) {
            cout << "| ";

            for (int col = 0; col < 9; col++) {
                cout << grid[row][col] << " ";

                if (col == 2 || col == 5 || col == 8) {
                    cout << "| ";
                }
            }

            cout << endl;

            if (row == 2 || row == 5 || row == 8) {
                cout << "+-------+-------+-------+" << endl;
            }
        }
    }

public:
    SudokuSolver() {
        solutionCount = 0;

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                grid[row][col] = 0;
                original[row][col] = 0;
            }
```

```
    }
}

bool loadPuzzle(const string& filename) {
    ifstream inputFile(filename);

    if (!inputFile) {
        return false;
    }

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            char cell;

            if (!(inputFile >> cell)) {
                return false;
            }

            if (cell == '_') {
                grid[row][col] = 0;
            }
            else if (cell >= '1' && cell <= '9') {
                grid[row][col] = cell - '0';
            }
            else {
                return false;
            }

            original[row][col] = grid[row][col];
        }
    }

    return true;
}

void printOriginalPuzzle() const {
    cout << "+-------+-------+-------+" << endl;
```

```
    for (int row = 0; row < 9; row++) {
        cout << "| ";

        for (int col = 0; col < 9; col++) {
            if (original[row][col] == 0) {
                cout << "_ ";
            }
            else {
                cout << original[row][col] << " ";
            }

            if (col == 2 || col == 5 || col == 8) {
                cout << "| ";
            }
        }

        cout << endl;

        if (row == 2 || row == 5 || row == 8) {
            cout << "+-------+-------+-------+" << endl;
        }
    }
}

void solve() {
    solutionCount = 0;

    if (!initialPuzzleIsValid()) {
        cout << "No solution found." << endl;
        return;
    }

    solveRecursive();

    if (solutionCount == 0) {
        cout << "No solution found." << endl;
    }
}
```

```
    int getSolutionCount() const {
        return solutionCount;
    }
};

int main() {
    // The five puzzle files are expected to be in the same directory
    // as the program.
    const string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (int i = 0; i < 5; i++) {
        SudokuSolver solver;

        cout << "========================================" << endl;
        cout << puzzleFiles[i] << endl;
        cout << "========================================" << endl;

        if (!solver.loadPuzzle(puzzleFiles[i])) {
            cout << "Could not open or read " << puzzleFiles[i] << "." << endl;
            cout << endl;
            continue;
        }

        cout << "Puzzle:" << endl;
        solver.printOriginalPuzzle();

        cout << endl;
        cout << "Solution(s):" << endl;

        solver.solve();
```

```
        if (solver.getSolutionCount() > 0) {
            cout << solver.getSolutionCount() << " solution(s) found." << endl;
        }

        cout << endl;
    }

    return 0;
}
```

Claude:

```
// Assignment4.cpp
// Recursive, object-oriented brute-force Sudoku solver (depth-first search
// with backtracking). Finds and prints ALL solutions for each puzzle file.

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
using namespace std;

class Sudoku {
private:
    static const int N = 9;
    int grid[N][N];          // 0 = empty
    int solutionCount;       // number of solutions found so far

    // Can 'val' be placed at (row, col) without breaking any rule?
    bool isValid(int row, int col, int val) const {
        for (int i = 0; i < N; i++) {
            if (grid[row][i] == val) return false;   // row
            if (grid[i][col] == val) return false;   // column
        }
        int br = (row / 3) * 3, bc = (col / 3) * 3;  // block
        for (int r = br; r < br + 3; r++)
            for (int c = bc; c < bc + 3; c++)
```

```
            if (grid[r][c] == val) return false;
    return true;
}

// Verify that the givens in the loaded puzzle do not conflict.
bool givensAreConsistent() {
    for (int r = 0; r < N; r++) {
        for (int c = 0; c < N; c++) {
            int v = grid[r][c];
            if (v == 0) continue;
            grid[r][c] = 0;               // temporarily remove to test
            bool ok = isValid(r, c, v);
            grid[r][c] = v;
            if (!ok) return false;
        }
    }
    return true;
}

// Find the next empty cell. Returns false if the grid is full.
bool findEmpty(int &row, int &col) const {
    for (row = 0; row < N; row++)
        for (col = 0; col < N; col++)
            if (grid[row][col] == 0) return true;
    return false;
}

// Recursive depth-first search with backtracking.
// Prints every complete solution it reaches and keeps searching.
void solve() {
    int row, col;
    if (!findEmpty(row, col)) {               // k == 81: solution found
        solutionCount++;
        cout << "Solution " << solutionCount << ":" << endl;
        print();
        return;
    }
    for (int val = 1; val <= 9; val++) {       // try each candidate
```

```
            if (isValid(row, col, val)) {
                grid[row][col] = val;
                solve();                    // recurse
                grid[row][col] = 0;             // backtrack
            }
        }
    }

public:
    Sudoku() : solutionCount(0) {
        for (int r = 0; r < N; r++)
            for (int c = 0; c < N; c++)
                grid[r][c] = 0;
    }

    // Read a puzzle from a file. Digits 1-9 are givens; '_' is blank.
    bool load(const string &filename) {
        ifstream in(filename.c_str());
        if (!in) return false;
        int count = 0;
        char ch;
        while (count < N * N && in.get(ch)) {
            if (ch >= '1' && ch <= '9')
                grid[count / N][count % N] = ch - '0', count++;
            else if (ch == '_' || ch == '0' || ch == '.')
                grid[count / N][count % N] = 0, count++;
            // anything else (spaces, newlines) is ignored
        }
        solutionCount = 0;
        return count == N * N;
    }

    void print() const {
        for (int r = 0; r < N; r++) {
            if (r > 0 && r % 3 == 0) cout << "------+-------+------" << endl;
            for (int c = 0; c < N; c++) {
                if (c > 0 && c % 3 == 0) cout << "| ";
                if (grid[r][c] == 0) cout << "_";
```

```
            else cout << grid[r][c];
            if (c < N - 1) cout << " ";
        }
        cout << endl;
    }
}

// Solve the loaded puzzle, printing all solutions found.
void solveAndReport() {
    solutionCount = 0;
    if (givensAreConsistent())
        solve();
    if (solutionCount == 0)
        cout << "No solution found" << endl;
    else
        cout << "Total solutions: " << solutionCount << endl;
}
};

int main() {
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";
        cout << "==============================" << endl;
        cout << filename << endl;
        cout << "==============================" << endl;

        Sudoku puzzle;
        if (!puzzle.load(filename)) {
            cout << "Could not read a valid puzzle from " << filename << endl << endl;
            continue;
        }
        cout << "Puzzle:" << endl;
        puzzle.print();
        cout << endl;
        puzzle.solveAndReport();
        cout << endl;
    }
    return 0;
```

}

e. Analyze the code in terms of correctness, execution time, space

complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness:

Claude:

Good: the code runs and works as given by the instructors. The consistent function indicates if the puzzle alread conflicts and it will stop for bad input. Can handle other blank makers

Bad: it does not account for any characters that are no specifically stated to handle. Also if you put and input that did not have many starting tokens and thus had a lot of possible solutions it would have a toon of prints

Chatgpt:

Good: the code runs and works as given by the instructors. The consistent function indicates if the puzzle already conflicts and it will stop for bad input. it does account for any characters that are no specifically stated to handle

Bad:it does not include any delimiter like claude so it is hard to differentiate between outputs. Connot handle other blank makers

Execution time:

Claude:

Good: it runs fast and recuses to speed the process as asked in the prompt

Bad: findEmpty() starts from 0,0 each time in the grid so if you could just start where you left off.

Chatgpt:

Good: : it runs fast and recuses to speed the process as asked in the prompt

Bad: : it runs fast and recuses to speed the process as asked in the prompt

Space complexity:

Claude:

Good: for space it is good it only has one grid at a time it prints then deletes so it doesn’t store any extra grids for solutions

Bad: I cant think of any improvements via space

Chatgpt:

Good: for space it is good it only has one grid at a time it prints then deletes so it doesn’t store any extra grids for solutions

Bad: this stores 2 grids rather than just one

Maintainability:

Claude:

Good: the code is clean and it contains botha a private and public interface. Each function is short and make easy sense of their purpose

Bad: no comments only documented headers. I also think in the load function that: grid[count / N][count % N] = ch - '0', count++; is very dense and very hard to understand for just storing a value as 0 so it could be written in 2 lines

Chatgpt:

Good: this code contains the names of puzzles stored as values so it would no be good if you wanted to add more puzzles you would have to add to the list of names.

Bad: no comments only documented headers. It also has the same code for both printing the original puzzle and the solution which it doesn't need. Also definitions are longer and less understandable

f. Select one to use as the basis for this assignment and justify why you picked it.

Each are basically identical on execution time. While GPT is stricter on input characters I think I will choose cluade because it outputs nicer there is less repetitive code and more understandable for me to edit.

g. Explain what you did to improve it in terms of correctness, execution time,

space complexity, and maintainability.

Correctness: I added a part that made sure if the file had any unidentified parts that it flagged it and returned false so that it only takes good input

Execution time: no changes

Space complexity: no changes

Maintainability: I have added detailed comments for every line, I also changed the grid[count / N][count % N] = ch - '0', count++; line to be 2 lines.

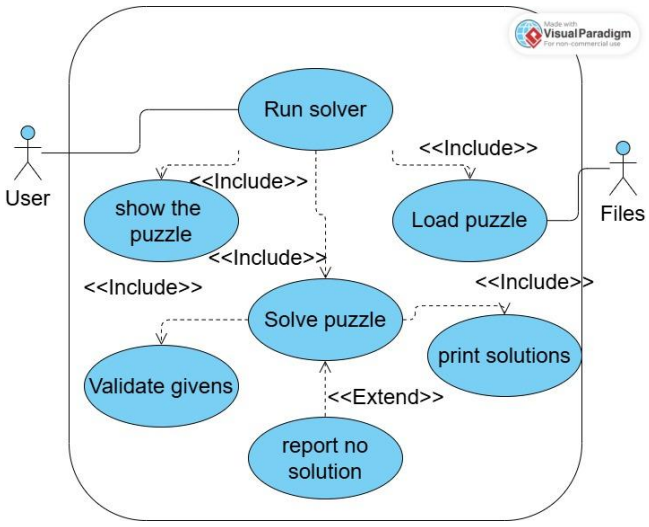
3. Make the changes identified in 2g above.

Added line by line comments on code and made changes to the Claude code with issues identified above

4. Compile and debug your program as described in the "C Basics" lecture.

5. Push both your source code, final cycle server executable file, and PDF of your

GenAI analysis to your GitHub repository.


Run solver
<<Include>>
<<Include>>
show the puzzle
Load puzzle
User
Files
<<Include>>
<<Include>>
<<Include>>
Solve puzzle
print solutions
Validate givens
<<Extend>>
report no solution